

Exercise 5: Univariate Optimization

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")  
remotes::install_version("ggplot2", "3.5.1")
```

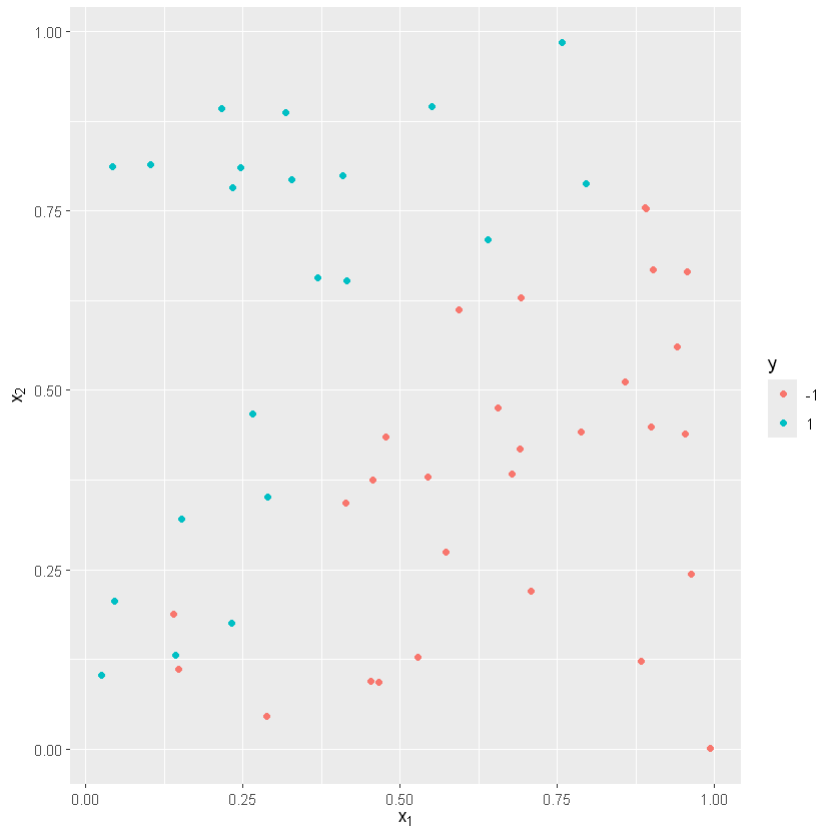
Setup

You are given the following data situation:

R

```
library(ggplot2)  
  
set.seed(123)  
  
X <- matrix(runif(100), ncol = 2)  
y <- -((X %*% c(-1, 1) + rnorm(50, 0, 0.1) < 0) * 2 - 1)  
df <- as.data.frame(X)  
df$type <- as.character(y)
```

```
ggplot(df) +
  geom_point(aes(x = V1, y = V2, color = type)) +
  xlab(expression(x[1])) +
  ylab(expression(x[2])) +
  labs(color = "y")
```



Python

```
import numpy as np
import matplotlib.pyplot as plt

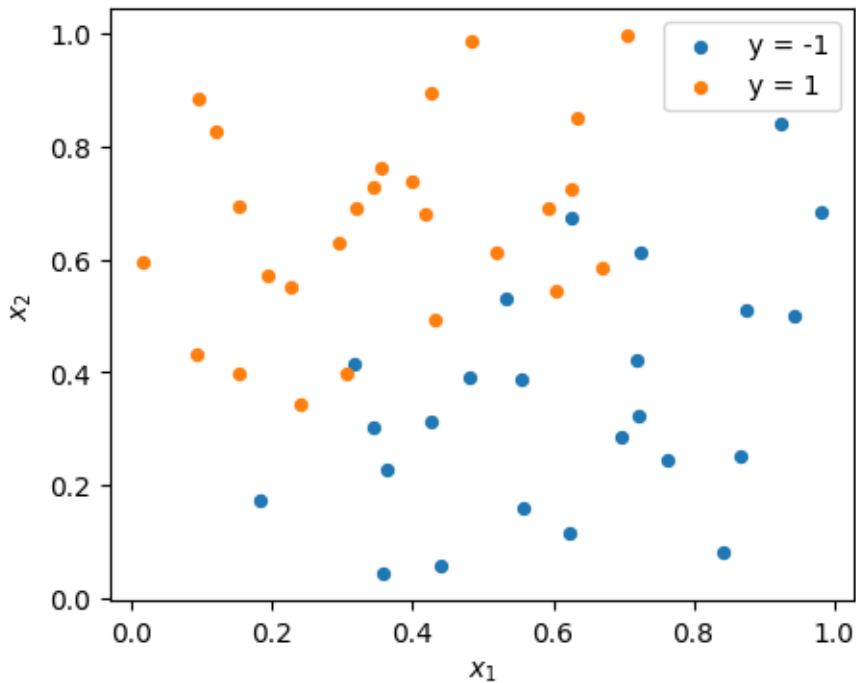
np.random.seed(123)

X = np.random.uniform(size=(50, 2))
lin = X @ np.array([-1.0, 1.0]) + np.random.normal(0, 0.1, 50)
y = -((lin < 0).astype(int) * 2 - 1)
```

```

fig, ax = plt.subplots(figsize=(5, 4))
for label in (-1, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
ax.set_xlabel(r"$x_1$")
ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()

```



i Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs. The data above (and consequently the optima below) differ between languages. Both implementations are correct; only the underlying data differs. When comparing R and Python, look at *algorithmic behavior* (convergence, bracket shrinking, zig-zag pattern) rather than expecting numerically identical θ values.

In the following we want to estimate a linear SVM without intercept and with $\lambda = 1$. We assume we know that $\theta_2 = 2$.

(a) Convexity of a 1D slice

Show that if $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ is convex then $g_c : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto f(x, c)$ for any $c \in \mathbb{R}$ is convex.

(b) Why the non-geometric primal SVM

Explain why the non-geometric primal linear SVM formulation should be used rather than the geometric one if we want to find θ_1 via the golden-ratio algorithm.

Note: We use this algorithm for educational purposes; in practice more advanced algorithms are typically used.

(c) Golden-section search

Find θ_1 via the golden-section algorithm. Implement the algorithm yourself. Use absolute error 0.01 as termination criterion and $[-3, 3]$ as the starting interval.

(d) Parabola interpolation through three points

Given three points $(x_1, y_1), (x_2, y_2), (x_3, y_3)$ show that the parameters $a, b, c \in \mathbb{R}$ of the interpolating parabola $p(x) = ax^2 + bx + c$ can be found via

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{pmatrix}^{-1} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}.$$

(e) Brent's method

Find θ_1 via Brent's method. Implement a simplified version (only check whether the proposed point is in the current interval) of the algorithm. Use absolute error 0.01 as termination criterion and $[-3, 3]$ as the starting interval. For the first step, use a golden-ratio step.

(f) Alternating univariate minimization

Now assume we do not know θ_2 . Initial guess is $\theta_2 = 0$. Alternately minimize w.r.t. either θ_1 or θ_2 via the golden-section method (starting interval reset to $[-3, 3]$ each time) while holding the other parameter fixed. Switch to the other parameter when absolute error is smaller than 0.01. Repeat this procedure 10 times.

(g) Optimization trace

How does the optimization trace of (f) look in parameter space?